

1차 방문(v4) 구역별 실행 절차

v4 분석 코드는 길 연결망 가운데 가장 큰 덩어리 하나만 남깁니다(lib/qnetwork.py ⑧).

5개 구는 서로 떨어진 네 묶음(종로·중 / 관악 / 광진 / 강서)이라, 한 번에 돌리면 한 묶음만 남고 나머지 구의 집은 통째로 빠집니다.

공개 도로 네트워크로 재 보면 가장 큰 덩어리만 남길 때 노드의 38.2%만 남습니다.

v4 코드는 반입 심의를 받은 그대로라 고칠 수 없으니, config.py 의 AREA_BBOX 만 바꿔 구역마다 따로 돌립니다. (v5 에서는 코드로 고침)

구역별 AREA_BBOX (EPSG:5186, 대상 구 범위 + 1,000m, 100m 단위로 바깥쪽 반올림)

구역	대상 구	config.py 에 넣을 줄	크기	들어 있는 선정지 (잘린 선까지 거리)
A	종로구·중구	AREA_BBOX = [194500, 548300, 203400, 560200]	8.9 × 11.9 km	송인동(2,068m), 신당동(1,949m), 대현산배수지공원 구간
B	관악구	AREA_BBOX = [190100, 536300, 200000, 545000]	9.9 × 8.7 km	봉천동(2,631m)
C	광진구	AREA_BBOX = [203900, 546100, 211100, 553700]	7.2 × 7.6 km	중곡동 무지개계단(1,881m)
D	강서구	AREA_BBOX = [178200, 546400, 190500, 557200]	12.3 × 10.8 km	화곡동(2,786m)

옮겨 적을 줄 (config.py 의 AREA_BBOX 줄을 구역마다 이 한 줄로 바꿈)

A AREA_BBOX = [194500, 548300, 203400, 560200]

B AREA_BBOX = [190100, 536300, 200000, 545000]

C AREA_BBOX = [203900, 546100, 211100, 553700]

D AREA_BBOX = [178200, 546400, 190500, 557200]

- 1,000m 를 넓힌 이유: 구 경계 너머의 병원·정류장까지 경로에 들어가게 하려고.
- v4 는 bbox 밖을 "데이터 없음"으로 보고, 잘린 선에서 500m(EDGE_BUFFER) 안의 건물을 통계에서 자동으로 뺍니다 (lib/qedge.py). 테스트 데이터에서 bbox 를 걸어 확인함: 기대 191채 = v4 표시 191채, 경계선 근처를 뺀 179채 모두 일치.
- 그래서 대상 구 안 건물은 잘린 선에서 1,000m 이상 떨어져 있어 빠지지 않습니다. 선정지 반경 300m 원도 잘린 선에서 모두 1,800m 이상이라 안전합니다.
- 넓힌 띠(대상 구 밖) 건물 일부는 결과에 들어가지만, 밖에서 tools/merge_exports.py 가 대상 구 안 격자·동만 남겨 합칩니다.
- 구역 bbox 끼리는 겹치지 않습니다 (A 는 x 203,400 까지, C 는 203,900 부터).

시작 전에 한 번: 국토정보필지 확인 (센터가 필지를 준 경우만)

1. config.py 의 DATA_ROOT_PARCEL = r"필지 폴더 경로" 입력 (서울 파일만 있는 폴더). 필지를 못 받았으면 None 그대로 두고 이 단계를 건너뛴 (그러면 run_all 이 06 을 자동으로 뺀)
2. python 01_inspect.py → 화면의 "JIMOK(지목) 예시: {...}" 줄 확인
 - 대, 전, 도, 답 같은 한글이 보이면 정상 (PARCEL_ENCODING = "UTF-8" 그대로)
 - ??? 나 Æ€ 처럼 깨지면 config.py 의 PARCEL_ENCODING = "CP949" 로 바꾸고 01 을 다시 돌려 확인
 - "필드 [...] ← PNU, EMD_CD, JIMOK 이 있어야 함" 에 세 이름이 모두 있는지도 확인

구역 하나 돌리는 순서 (A → B → C → D 로 네 번): python run_all.py 한 줄

1. config.py 의 AREA_BBOX = ... 줄을 위 표의 값으로 바꾸고 저장. (QGIS 콘솔에서는 run_all 을 쓸 수 없으니 OSGeo4W Shell 에서 실행)
2. python run_all.py

- 화면에 ##### 01_inspect.py ##### 부터 01, 02, 03, 04, 05, 06 순서로 단계 이름이 나타남. 06 은 DATA_ROOT_PARCEL 에 필지 경로가 있을 때만 (없으면 05 에서 끝). SKT·KCB 경로(JOIN_DATA)를 넣었으면 07 도 자동으로 이어짐
- 끝나면 맨 아래에 ...\hbi> 가 다시 나타남. 한 단계에서 오류가 나면 거기서 멈추고 !! <단계>.py 에서 중단. 위 오류 메시지를 확인하세요. 가 나옴 → 아래 "막혔을 때"
- 5줄로 따로 돌린 것과 결과가 같음 (리허설에서 output CSV·PNG md5 동일, gpkg 는 저장 시각만 다름). 06 은 04·05 결과를 바꾸지 않아서 마지막에 돌아도 됨

3. 끝난 뒤 창을 위로 올려 확인할 것

- 02: "최대 연결망 노드 비율" 이 90% 이상인지. 이 구역만 돌리면 90% 이상이 정상. 한참 낮으면 그 구역 안에서도 연결망이 갈라진 것 → 메모(구역 이름 + 비율)만 하고 계속
- 03: "네트워크 연결 N%" 가 95% 이상인지
- 03: "데이터 경계 500m 이내 건물 N개 → 통계에서 제외 예정 (x%)" 는 넓힌 때 때문에 평소보다 높게 나옵니다. 이때는 EDGE_BUFFER 를 줄이지 마세요 (README 의 "50% 넘으면 300으로" 는 구역별 실행에는 적용하지 않음)
- 05: 완료 → output/ (반출 신청 대상)
- 06 (필지를 받았을 때): "필지 N개" 와 "그중 지목 '대' 필지 수" 가 0 이 아닌지. "지목 '대' 필지가 0개입니다" 가 나오면 위의 PARCEL_ENCODING 확인으로 돌아감. 06 은 이 구역 건물 범위 안의 필지만 읽습니다
- 04 의 선정지·구간은 5곳·전 구간을 매번 계산해도 됨 (합칠 때 그 구역 것만 씀)

4. (시간 되면) python 08_sensitivity.py (민감도 분석. run_all 에는 --sens 를 붙여야만 들어가므로 따로 한 줄)

5. output 폴더 이름을 output_A 로 바꾸기 (탐색기에서 이름 바꾸기, 또는 명령창 ren output output_A)

- 복사가 아니라 이름 바꾸기: 다음 구역을 돌릴 때 output 이 새로 비어 있어야 앞 구역 파일이 섞이지 않습니다
- work 폴더는 그대로 두면 다음 구역이 덮어씁니다 (반출하지 않음)

네 구역을 다 돌린 뒤 AREA_BBOX = None 으로 되돌려 둡니다.

구역	02 연결망 %	03 네트워크 연결 %	03 경계 제외 %	06 필지 수 / '대'	output_? 만듦	메모
A 종로·중					[]	
B 관악					[]	
C 광진					[]	
D 강서					[]	

막혔을 때

- run_all 이 중간에 멈추면, 원인을 고친 뒤 멈춘 단계부터 한 줄씩 다시 칠 수 있음 (앞 단계 결과는 work 폴더에 남아 있음). 순서는 run_all 과 같음:

python 02_network.py → python 03_hbi.py → python 04_validate.py → python 05_export.py → (필지를 받았을 때) python 06_parcel.py

- 예: 05 에서 멈췄으면 05 → 06 만. 단, config 의 AREA_BBOX 나 데이터 경로를 고쳤으면 02 부터(또는 python run_all.py 로 처음부터) 다시
- 05 에서 min() iterable argument is empty → 그 구역의 유효 건물이 0 개. 03 화면의 '경계 N m 이내 건물' 비율과 AREA_BBOX 붙여넣기를 확인. 안 풀리면 그 구역은 건너뛰고 다음 구역으로.
 - 확인할 것: 03 의 "데이터 경계 500m 이내 건물 N개 (x%)" 가 100% 이면 모두 제외된 것. AREA_BBOX 네 숫자가 위 표와 같은지(순서 x최소, y최소, x최대, y최대), 씬표·대괄호가 빠지지 않았는지
 - 건너뛴 때: 멈춘 구역의 output 에는 04 결과 일부만 남아 있으니 ren output output_A_실패 처럼 이름을 바꿔 두고(반출 안 함) 메모 칸에 적은 뒤 다음 구역으로

- EDGE_BUFFER 를 줄여서 풀지 않습니다 (맥 리허설에서만 100 으로 썼음)

반출 신청

- output_A, output_B, output_C, output_D 네 폴더 (모두 격자·행정동 집계, 건물 5개 미만 제외 — v4 반출 규칙 그대로)
- 걸리는 시간은 구역마다 다름. 전체를 한 번에 돌릴 때보다 구역이 작아 02:03 은 오히려 빠를 수 있음

받은 뒤 (밖에서)

```
# 1) 네 폴더를 그대로 넣기
results/raw_export_zones/A/ ← output_A 의 파일들
results/raw_export_zones/B/ ← output_B ...

# 2) 하나로 합치기 → results/raw_export
python3 tools/merge_exports.py --zone A=results/raw_export_zones/A --zone B=results/raw_export_zones/B \
    --zone C=results/raw_export_zones/C --zone D=results/raw_export_zones/D

# 3) 고령인구 붙이기 (레포 최상위 도구)
python3 tools/outside_elderly.py results/raw_export/dong_hbi.csv

# 4) 기획서
cd deck && npm run build:real
```

합칠 때 무엇을 어떻게 근사했는지는 results/raw_export/merge_notes.txt 에 적힙니다.

- summary 는 대상 구 격자로 다시 셉니다. 건물 5개 미만 격자는 원래 반출에 없으므로 그만큼 적게 셉니다. 중앙값은 합칠 수 없어 빠집니다.
- 선정지 백분위와 신규 후보는 합친 격자 전체 기준으로 다시 계산합니다.
- 기여도·민감도는 구역 값을 분석 건물 수로 가중 평균한 근사입니다.
- 구간(validation_routes)은 출발·도착이 그 구역 bbox 안, 잘린 선에서 500m 이상 안쪽인 구역 값만 씁니다.